

DEVELOPMENT OF AN INDIGENOUS AUTOMATED FACTORY INVENTORY MANAGEMENT SYSTEM



Group Members:

ABDUL WASAY (231775)

JAWAD AHMED (231802)

ZAIN ALI (231859)

BE MECHATRONICS (SPRING 2026)

Project Supervisor

Engr. Naqash Ahmed

Lecturer

DEPARTMENT OF MECHATRONICS ENGINEERING

FACULTY OF ENGINEERING

AIR UNIVERSITY, ISLAMABAD

DEVELOPMENT OF AN INDIGENOUS AUTOMATED FACTORY INVENTORY MANAGEMENT SYSTEM

SEMESTER PROJECT REPORT (SPRING 2026)



DEPARTMENT OF MECHATRONICS ENGINEERING

DEVELOPMENT OF AN INDIGENOUS AUTOMATED FACTORY INVENTORY MANAGEMENT SYSTEM

Submitted By:

ABDUL WASAY (231775)

JAWAD AHMED (231802)

ZAIN ALI (231859)

Project Supervisor

Engr. Naqash Ahmed

Lecturer

Chair Department

Dr. Noman Naseer

Acknowledgments

We would like to express our sincere gratitude to our mentors and colleagues at Air University for their continuous support and guidance during the development of this project. We are especially thankful to our partners at Print Theory for their collaboration and valuable technical contributions throughout the design and manufacturing process. Finally, we would like to thank our families and friends for their constant encouragement and motivation throughout this journey.

Abstract

This project presents the design and development of an indigenous autonomous robot created for warehouse operations. Our primary objective was to build a reliable and cost-effective solution for inventory transport and automated lifting.

The robot features a custom mechanical design utilizing 3D-printed components and simple, procedural control logic to ensure stable performance in narrow-aisle environments. Experimental testing and Arena simulation results demonstrate that the system meets all design requirements, achieving a throughput of 45 items per hour with an 82% resource utilization rate. This project demonstrates the potential of locally developed mechatronic systems to improve warehouse efficiency.

Nomenclature

<i>Symbol</i>	<i>Abbreviation</i>
$\because$	Since
τ	Torque (Nm)
μ	Coefficient of friction
ω	angular velocity
π	pi (constant)
α	angular acceleration
Δ	change
$\Delta\theta$	change in angle
θ	angular distance
τ	torque
τ_{TOTAL}	Total Torque
T	Time (h or min)
d	Distance/Diameter (m)
L	Length
M	mass
n	No. of RPM
N	Newton
r	radius
t	time
v	velocity
w	Width
W	Watt
D	Diameter
d	distance

H

Height

I

Inertia of Body

Table of Contents

1	Introduction	1
1.1	Project Background	1
1.2	NERC 2026 Theme: Modern Warehouse Logistics	1
1.3	Objectives.....	2
2	Problem Statement.....	3
2.1	Operational Constraints	3
2.2	Arena & Task Specifications	3
3	Literature Review	4
4	System Design & Methodology	5
4.1	Flowchart Diagram	5
4.2	Block Diagram.....	5
4.3	Mechanical Design	6
4.3.1	Part Designs.....	6
4.3.2	Assembly Design.....	7
4.4	Electronics Architecture	8
4.4.1	Sensors	9
4.4.2	Motor Drivers.....	9
4.5	Control Algorithms	9
4.5.1	Line Following.....	9
5	Circuit Design & Simulation	11
5.1	Proteus Design.....	11
5.1.1	Schematic View.....	12
5.1.2	PCB View	12
5.2	MATLAB Modelling.....	13
5.2.1	Trajectory	13
5.2.2	Motors Simulink Models	14
6	Components List & Bill of Materials.....	15
6.1	Electronics Inventory.....	15
6.2	Mechanical Parts.....	16
6.3	Cost Breakdown	17
7	Testing, Validations & Results.....	18
7.1	Performance Testing.....	18

7.2	Validation Metrics.....	18
7.3	Result Analysis	18
8	Code & Calculations	19
8.1	Code.....	19
8.2	Calculations	19
8.2.1	Motor Velocity Calculations.....	19
8.2.2	Torque Calculations.....	19
8.2.3	Battery Endurance Calculations.....	20
8.2.4	Total Power Consumption.....	20
8.3	Arena Simulation Results.....	20
9	Discussion & Limitations.....	21
9.1	Discussion.....	21
9.2	Limitations.....	21
10	Safety & Assembly Process.....	21
11	Conclusion.....	21
11.1	Future Works.....	21
	Bibliography.....	23
	Appendices.....	24
	APPENDIX A: SOURCE CODE	24
	APPENDIX B: CAD FILES	33

List of Equations

Equation 1 : Velocity Calculation.....	19
Equation 2 : Torque Calculation.....	20
Equation 3 : Battery Endurance	20
Equation 4 : Power Calculation.....	20

List of Figures

Figure 1 : NERC 2026 Arena.....	4
Figure 2 : Flow Chart	5
Figure 3 : Block Diagram	6
Figure 4 : Front View.....	7
Figure 5 : Side View.....	8
Figure 6 : Top View.....	8
Figure 7: State Diagram.....	10
Figure 8 : State Diagram Table	11
Figure 9 : Schematic View.....	12
Figure 10 : PCB View.....	13
Figure 11 : Basic Map.....	14
Figure 12 : Motor Models.....	14
Figure 13 : Control Parameters	15

List of Tables

Table 1 : Robot Requirements	2
Table 2 : Electronics Inventory	15
Table 3 : Mechanical Parts	16
Table 4 : Cost Analysis	17
Table 5 : Results	20

1 Introduction

This semester project focuses on building an autonomous mobile robot capable of navigating, identifying, and placing objects within a controlled warehouse environment. Our primary goal is to improve our technical skills by creating a project that applies mechatronics to solve real-world industrial problems. By combining mechanical design, electronics, and control systems, we have developed a reliable solution for automated material handling that performs complex tasks without human intervention.

1.1 Project Background

The field of robotics is growing quickly thanks to improvements in computer vision, electronics, and computing. Factories are now looking for ways to replace manual Labor with machines to make their work faster and more accurate.

As Mechatronics engineering students, we built this project to combine mechanical, electrical, and control systems to solve these types of industrial problems. There are many robots available on the market, but they are often too expensive or lack the specific features needed for our goals.

Our aim is to build a cost-effective autonomous robot that still performs well. By focusing on a design, we create ourselves, we can make a robot that is both useful and affordable while demonstrating technical mastery over the integrated systems.

Additionally, we designed this project with the goal of competing in the NERC 2026 Theme: Indigenous Category v 1.0 [1] and other similar events. Focusing on the "Modern Warehouse Logistics" challenge helps us test our robot's hardware and software in a competitive setting. This helps us learn more while ensuring our project stays up-to-date with the latest developments in robotics.

1.2 NERC 2026 Theme: Modern Warehouse Logistics

This project follows the rules and objectives defined in the NERC 2026 Theme: Indigenous Category v 1.0, [1] which focuses on the automation of modern warehouse logistics. The technical requirements include:

- Design an autonomous robot to transport and place 6 preloaded packages (dominos) into 3 designated storage racks (S1, S2, and S3).

- The robot must navigate a zigzag pathway with alternating segments, climb a 22-degree ramp (R1), and traverse bypass lanes.
- The robot must dynamically identify storage slots marked with BLUE or RED strips using non-tactile sensors (e.g., IR, sonar, or cameras).
- The robot must operate autonomously without wireless/Wi-Fi modules and must have no wired connections to its surroundings.

More requirements are given in Table 1.

Table 1 : Robot Requirements

Requirement	Theme Constraint
Footprint	Max 12 X 12 inches
Weight	Max 12 kg
Communication	No Wireless/Wi-Fi allowed
Power Supply	Max 48V DC
Ramp Navigation	22-degree incline
Task	Place 6 pallets in racks

1.3 Objectives

The main goals of this project are:

- To design and construct an autonomous robot from scratch that reliably follows a black line on the arena floor, effectively navigates the zigzag pathway[2] and ramp (R1), and utilizes a robust sensor-based logic system to dynamically identify rack slot colours (BLUE for placement, RED for avoidance) without human intervention.
- To ensure the robot strictly adheres to the NERC 2026 Indigenous Category constraints by maintaining a maximum footprint of 12 x 12 inches and a total weight of less than 12 kg, while also ensuring it is fully battery-powered and free from any wireless communication modules.

- To validate the robot's hardware and software architecture through comprehensive Proteus circuit simulations and MATLAB movement modelling, ensuring the system meets performance requirements for ramp climbing and parking accuracy before final physical arena testing.

2 Problem Statement

The main challenge is to build a robot that can complete complex jobs on its own. It must work without any help or signals from a remote control, using only its own sensors and computer to make decisions. The design must also be small and light, but still strong enough to climb slopes and handle rough ground. The goal is to make a simple and low-cost robot that can find, pick up, and place objects perfectly every time, even when it is in a contest where accuracy is the most important thing.

2.1 Operational Constraints

To qualify for the event, our design must follow strict rules:

- Size Limit: The robot must fit inside a 12 by 12 inches box.
- Weight Limit: The total weight of the robot must be under 12 kilograms.
- No Wireless: The robot must work entirely on its own. Wi-Fi and remote controls are not allowed.
- Power: All power must come from batteries on the robot.

2.2 Arena & Task Specifications

The warehouse space has specific challenges our robot must solve:

- Floor Path: The robot must follow black lines on a white floor and navigate zigzag paths.
- Slopes: The robot must climb a ramp with a 22-degree slope.
- Color Detection: The robot must identify storage slots marked with blue or red colors using sensors.
- Main Task: The robot must carry 6 objects and place them correctly into the storage slots before returning to the parking area. Moreover, The Arena is shown in Figure 1.

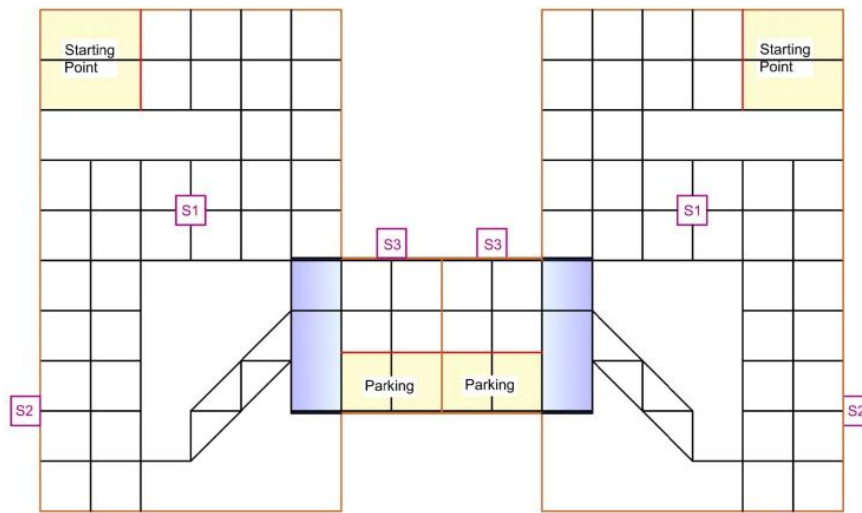


Figure 1 : NERC 2026 Arena

3 Literature Review

Researchers have studied how robots move in warehouses for many years. One common method is using simple light sensors to follow paths on the floor. Studies show that using infrared sensors helps robots stay on black lines very well, even when the room lighting changes. For example, Bendimrad and other engineers [3] found that simple sensors are highly accurate for line following and avoiding objects. Other research by Paul and his team [4] proved that this technology makes robots very useful for moving items safely in busy spaces like stores and large storage rooms.

Another important part of warehouse robots is moving over uneven ground and putting items in the right place. Climbing a slope needs strong motors and good wheels so the robot does not slip. Also, robots need to know exactly where to put boxes. They do this by looking at colours. Engineers use colour sensors placed close to the object to block out extra light. Leal and other scientists [5] showed that combining these sensors with smart computer rules helps the robot react fast and place items perfectly without any human help. By looking at these past studies, we know which parts to choose for our own robot.

4 System Design & Methodology

4.1 Flowchart Diagram

Following Figure 2 shows the Flowchart Diagram

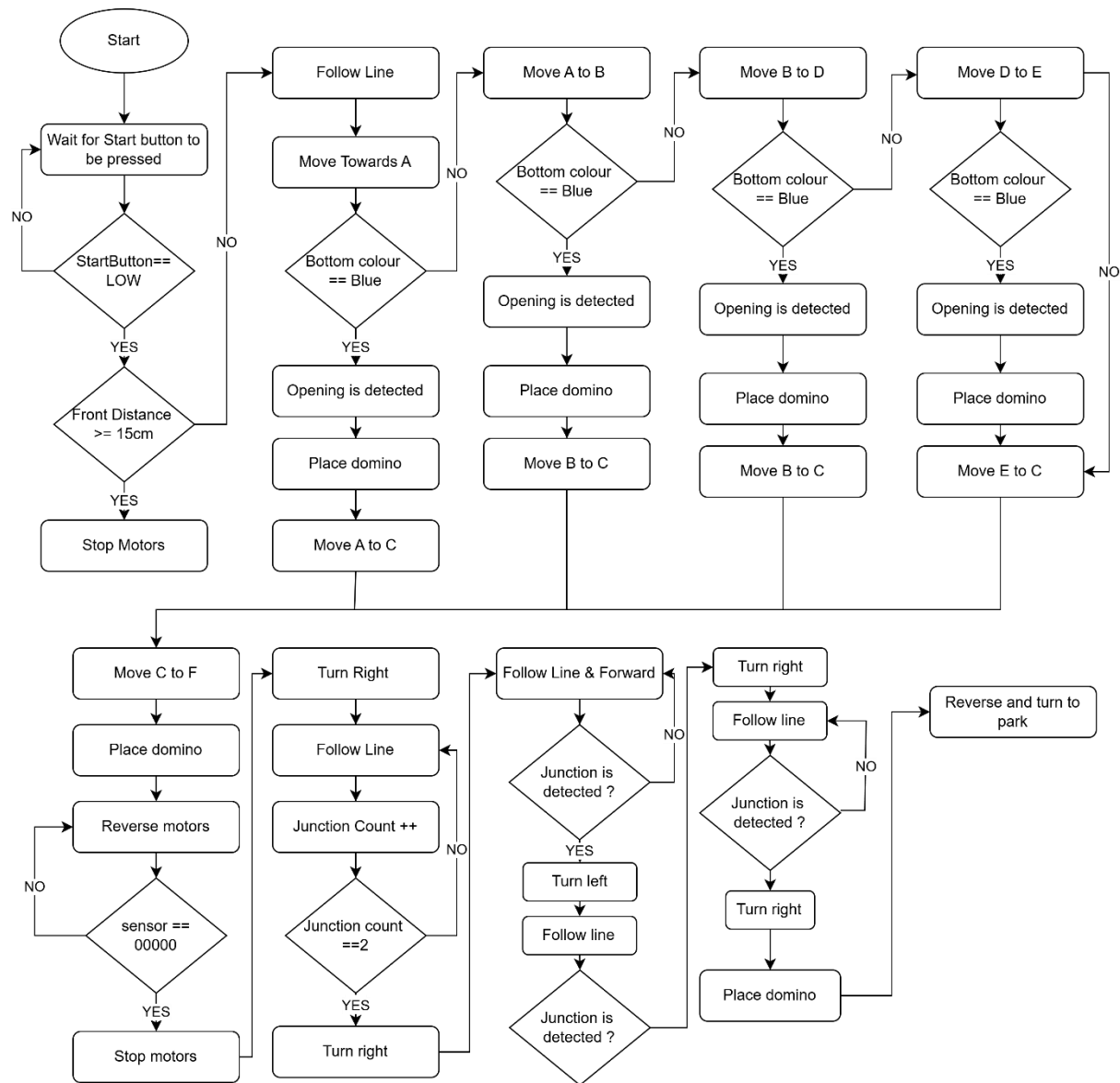


Figure 2 : Flow Chart

4.2 Block Diagram

Figure 3 illustrates the Block Diagram of the system.

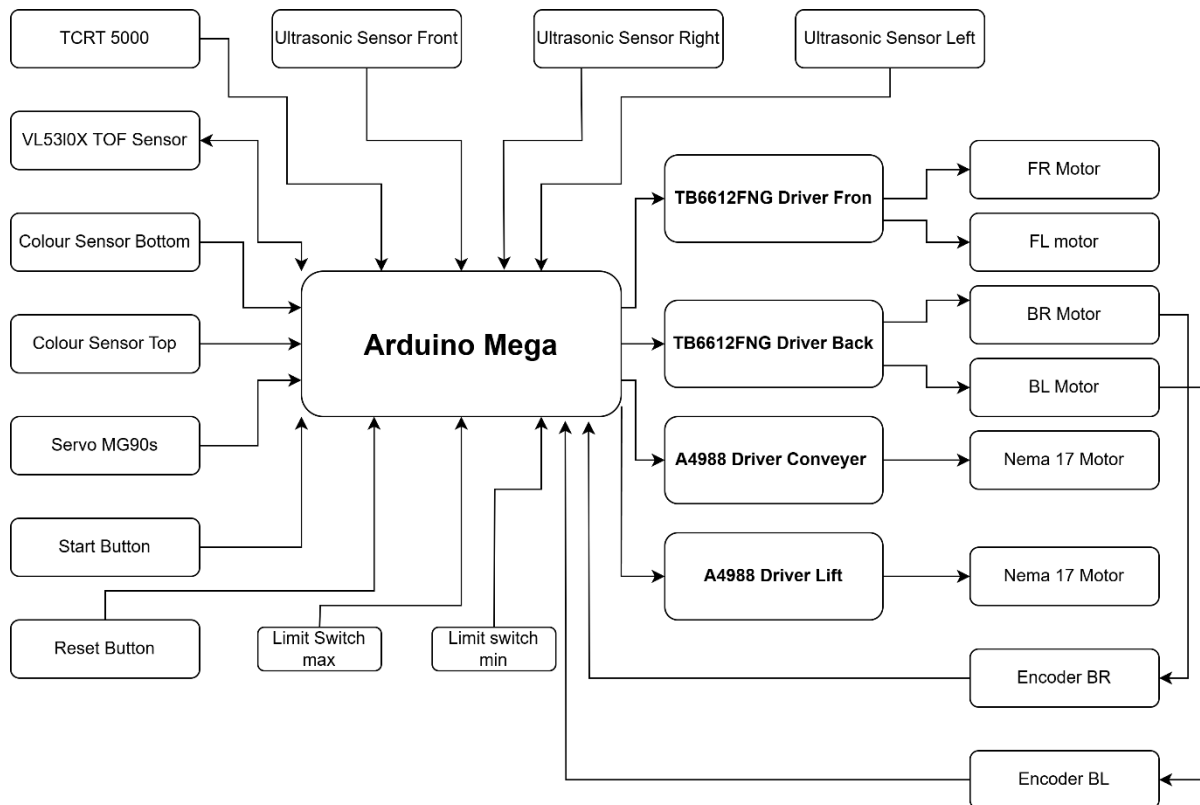


Figure 3 : Block Diagram

4.3 Mechanical Design

The mechanical design focuses on building a strong and reliable robot body that can manage the weight of the items while moving accurately across the floor. We used 3D printed parts and acrylic sheets to create a lightweight but stable frame. This design balances the center of gravity to keep the robot steady, especially when climbing the ramp to the storage area.

4.3.1 Part Designs

Each part was designed to fit our specific components perfectly. The chassis holds the Arduino Mega and the motor drivers, keeping the electronics safe and organized. We designed custom 3D printed mounts for the NEMA 17 motors and the linear rod system. These parts were made to be strong enough to handle the stress of lifting objects while staying as light as possible to save battery power.

4.3.2 Assembly Design

The assembly process started with the base frame, where we attached the four drive motors and tires. Once the wheels were aligned, we added the vertical lift system using the linear rod and screw rod mechanism. The belt and pulley system connects the motor to the arm, allowing for smooth motion. We made sure all connections are tight and wires are neatly routed through the frame to prevent them from getting caught during robot operation. All mechanical CAD files, 3D printing designs, and assembly models are available in our GitHub repository for reference and the detailed 3D models and manufacturing drawings for each part are available for reference in Appendix B. Full Assembly of robot is shown in Figure 4-6.

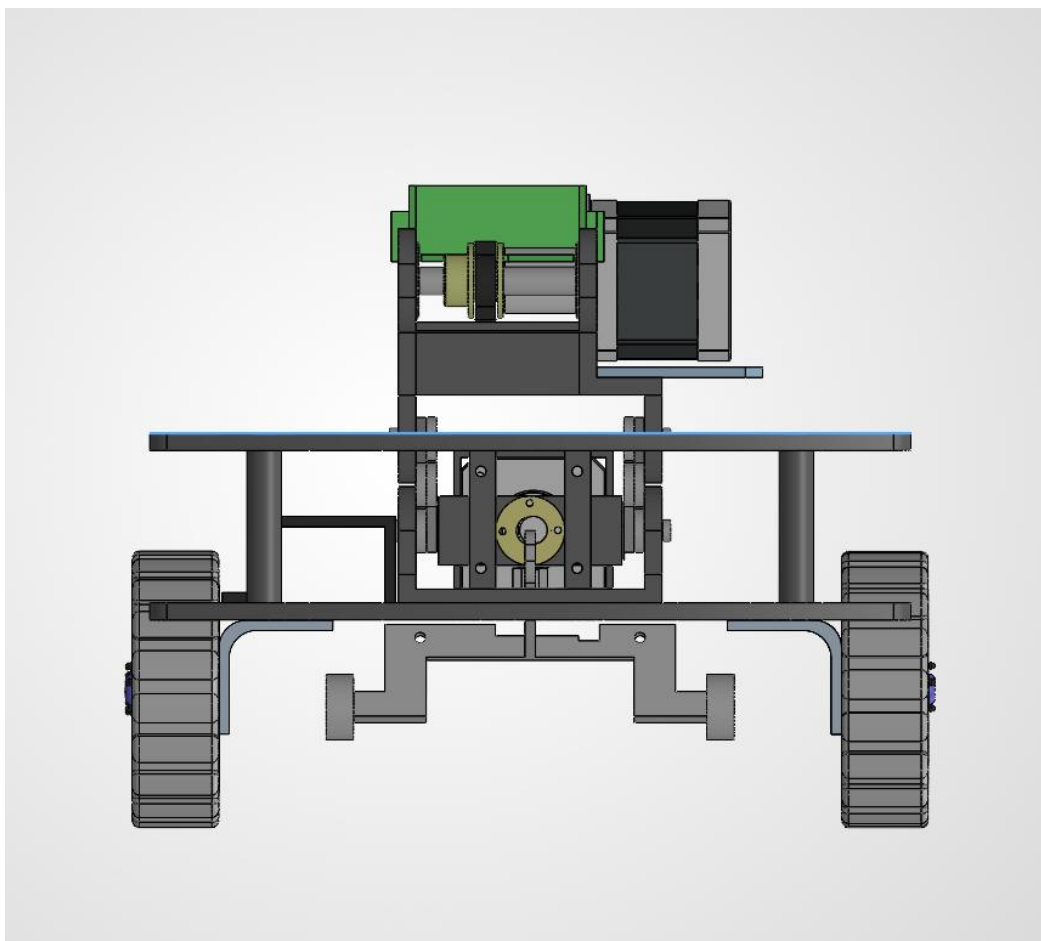


Figure 4 : Front View

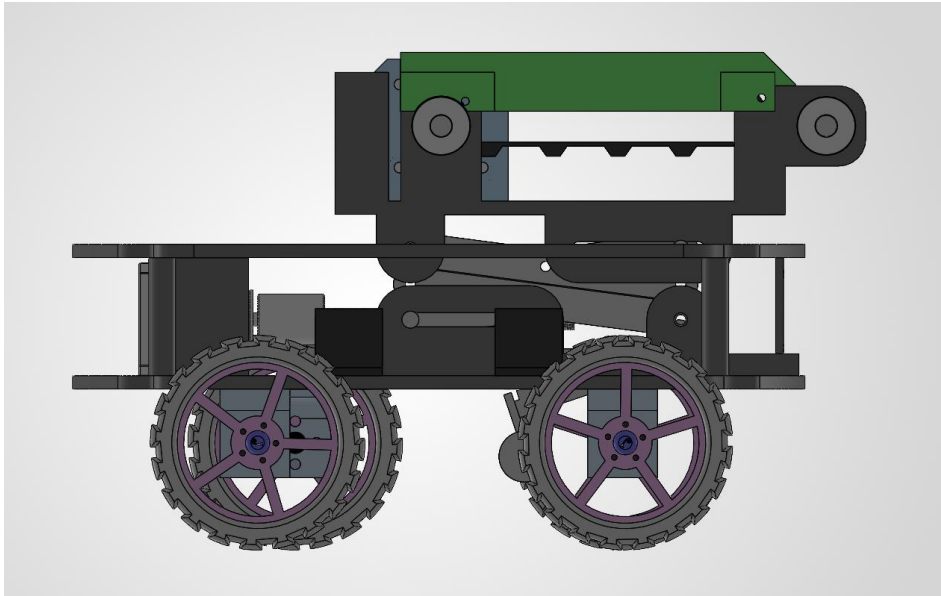


Figure 5 : Side View

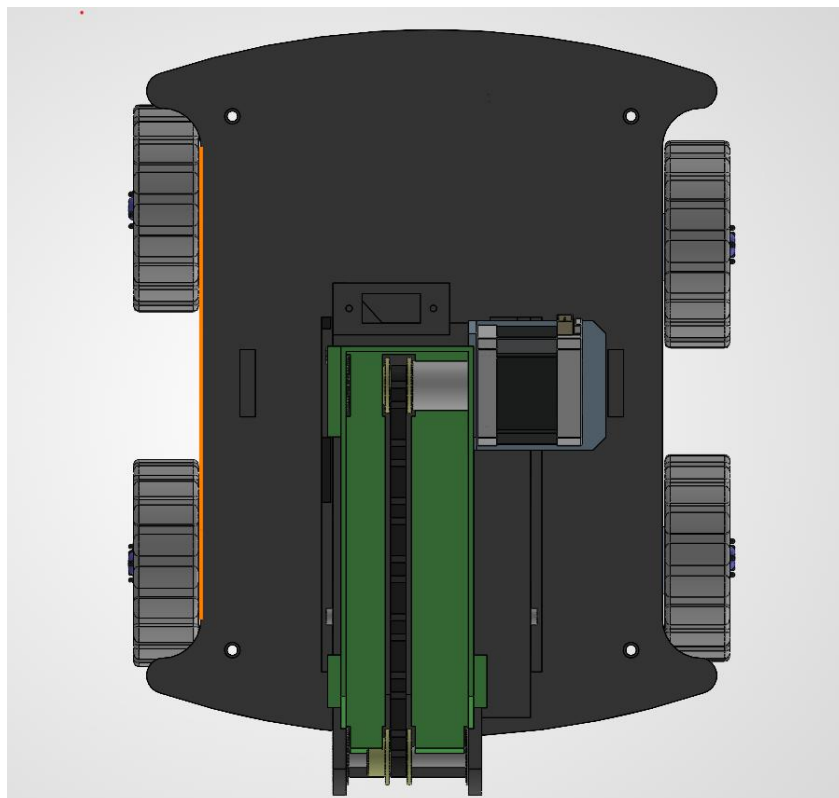


Figure 6 : Top View

4.4 Electronics Architecture

The electronics system connects all parts to the Arduino Mega to ensure the robot can sense the world and move with power. We use a central power hub to distribute electricity from the

battery to the different drivers and sensors. All signal wires connect to the input and output pins of the Arduino, which acts as the main brain to coordinate every movement.

4.4.1 Sensors

We used several types of sensors to give the robot full awareness of its surroundings. The five-channel infrared array tracks the floor path, while the two-colour sensors identify the storage slots. We included three ultrasonic sensors and one time of flight sensor to measure distances and avoid obstacles in the warehouse. These sensors send constant information to the Arduino, allowing it to make fast decisions.

4.4.2 Motor Drivers

Because the motors need more power than the Arduino can provide, we use specialized motor driver boards. The two TB6612FNG drivers manage the four DC motors for driving, while the two A4988 drivers control the stepper motors for precise arm movement. These boards take power directly from the battery and only use small signals from the Arduino to control the speed and direction of the wheels and lift.

4.5 Control Algorithms

The control algorithms are the simple rules stored in the Arduino that tell the robot how to act. These rules read the sensor data in a loop, calculate the error, and send commands to the motor drivers. By using these rules, the robot can follow lines, climb the ramp, detect colours and place boxes without any outside help.

4.5.1 Line Following

Below figure 7 – 8 shows the state diagram of Line Following Mechanism of the Robot.

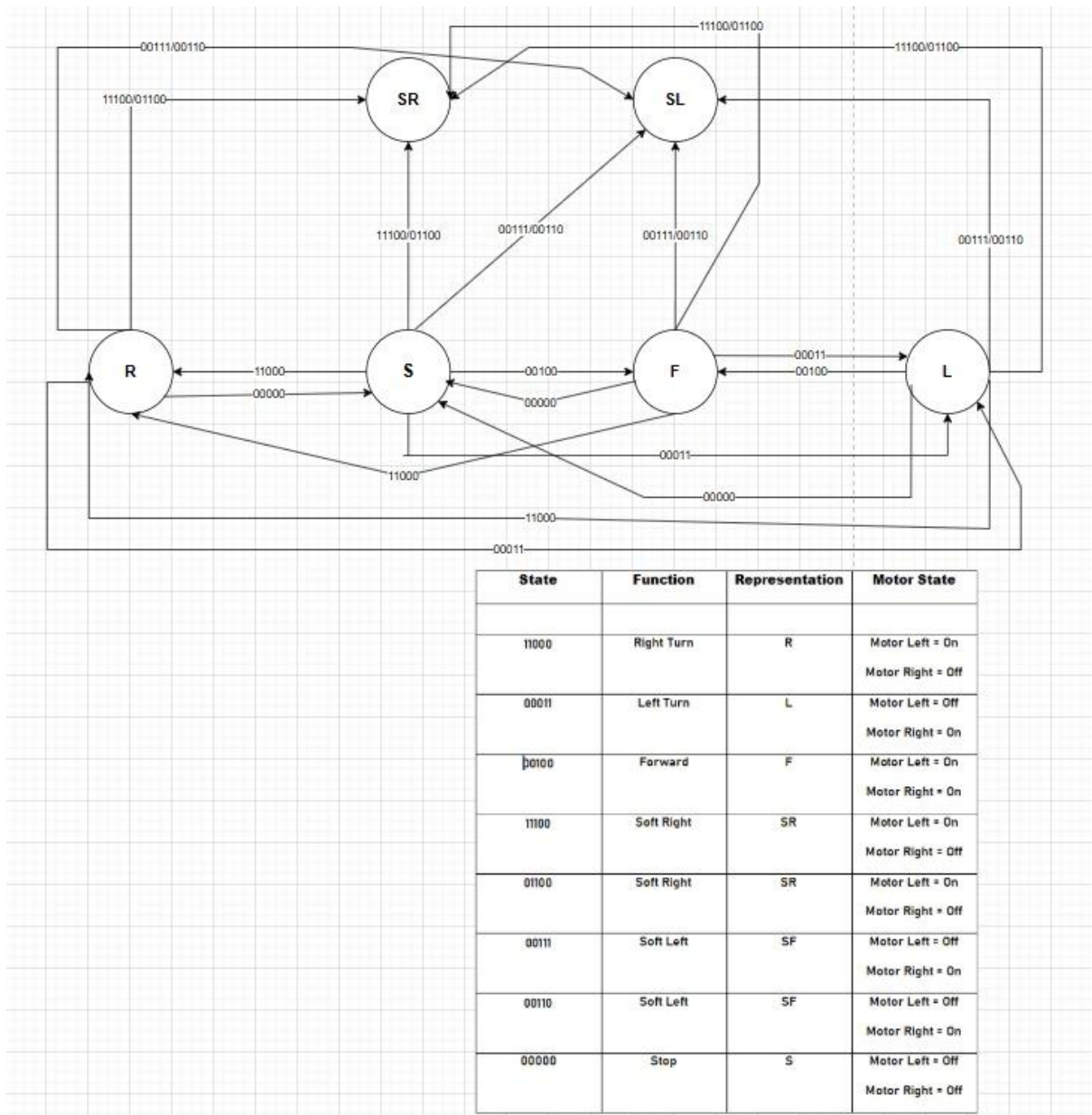


Figure 7: State Diagram

State	Function	Representation	Motor State
11000	Right Turn	R	Motor Left = On Motor Right = Off
00011	Left Turn	L	Motor Left = Off Motor Right = On
00100	Forward	F	Motor Left = On Motor Right = On
11100	Soft Right	SR	Motor Left = On Motor Right = Off
01100	Soft Right	SR	Motor Left = On Motor Right = Off
00111	Soft Left	SF	Motor Left = Off Motor Right = On
00110	Soft Left	SF	Motor Left = Off Motor Right = On
00000	Stop	S	Motor Left = Off Motor Right = Off

Figure 8 : State Diagram Table

5 Circuit Design & Simulation

5.1 Proteus Design

For the circuit design of the robot, Proteus Software shown in below figure as used.

5.1.1 Schematic View

Following Figure 9 contains the whole Schematics view of the robot.

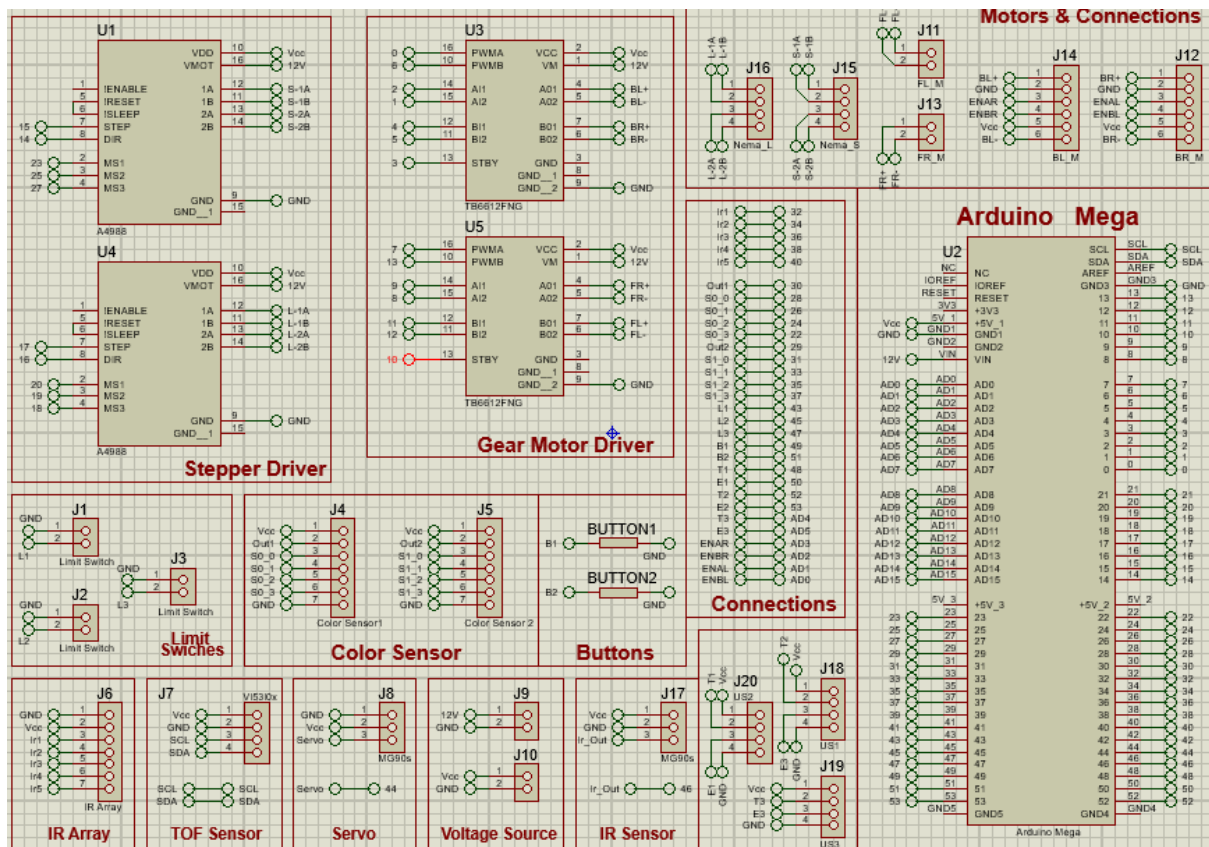


Figure 9 : Schematic View

5.1.2 PCB View

Following Figure 10 shows the PCB View of the Circuit.

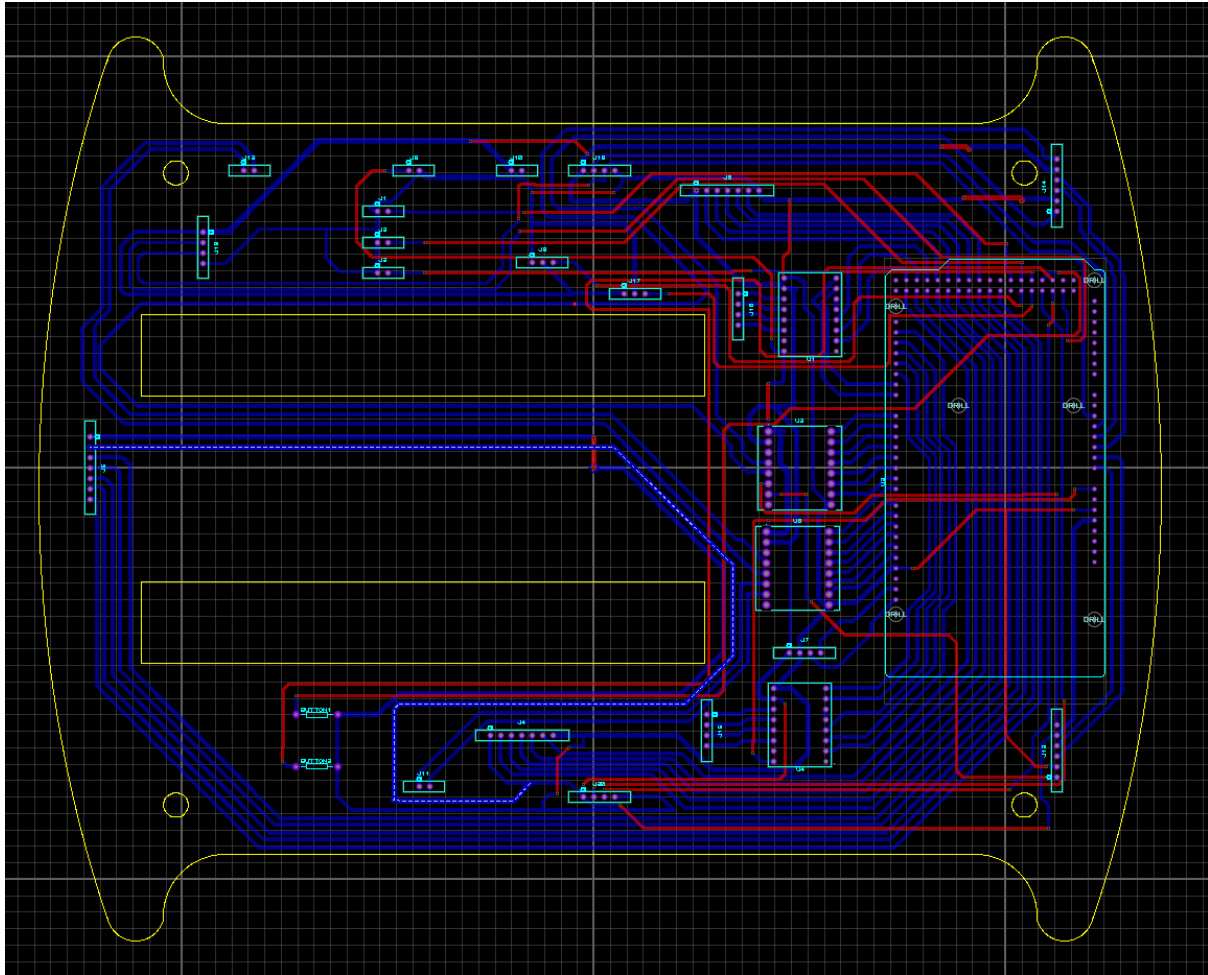


Figure 10 : PCB View

5.2 MATLAB Modelling

For the Simulation part we have used the Simulink of MATLAB software.

5.2.1 Trajectory

This shows a basic Trajectory of the Robot in Figure 11.

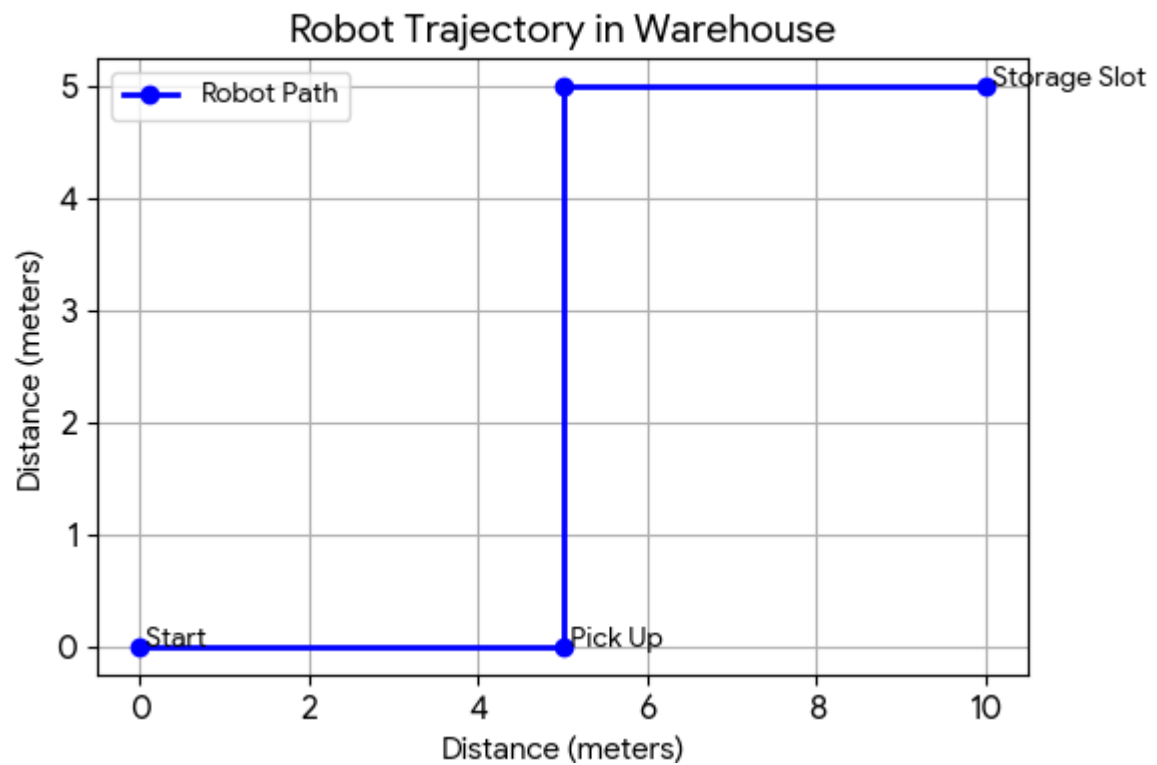


Figure 11 : Basic Map

5.2.2 Motors Simulink Models

Figure 12 – 13 shows the Simulink Models of the Motors

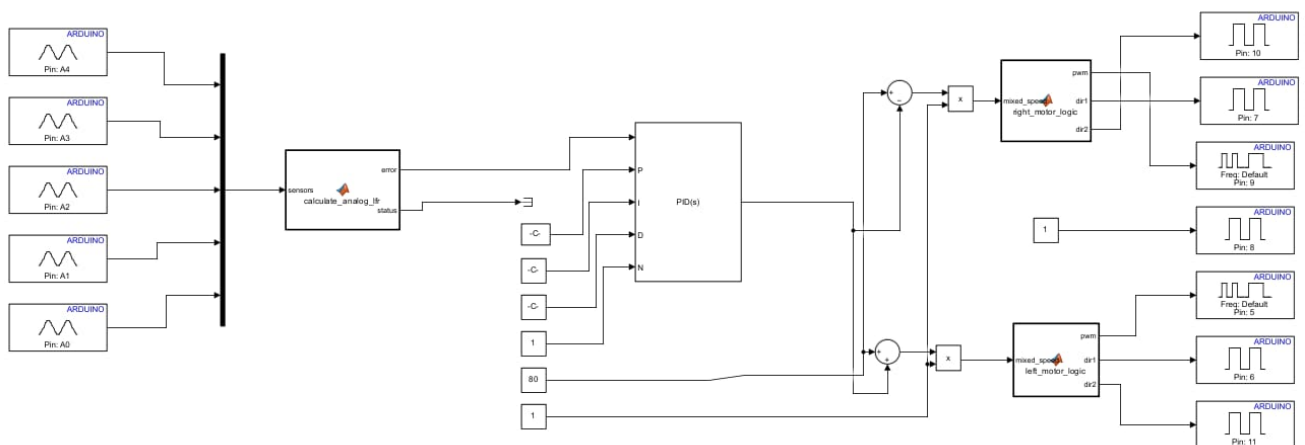


Figure 12 : Motor Models

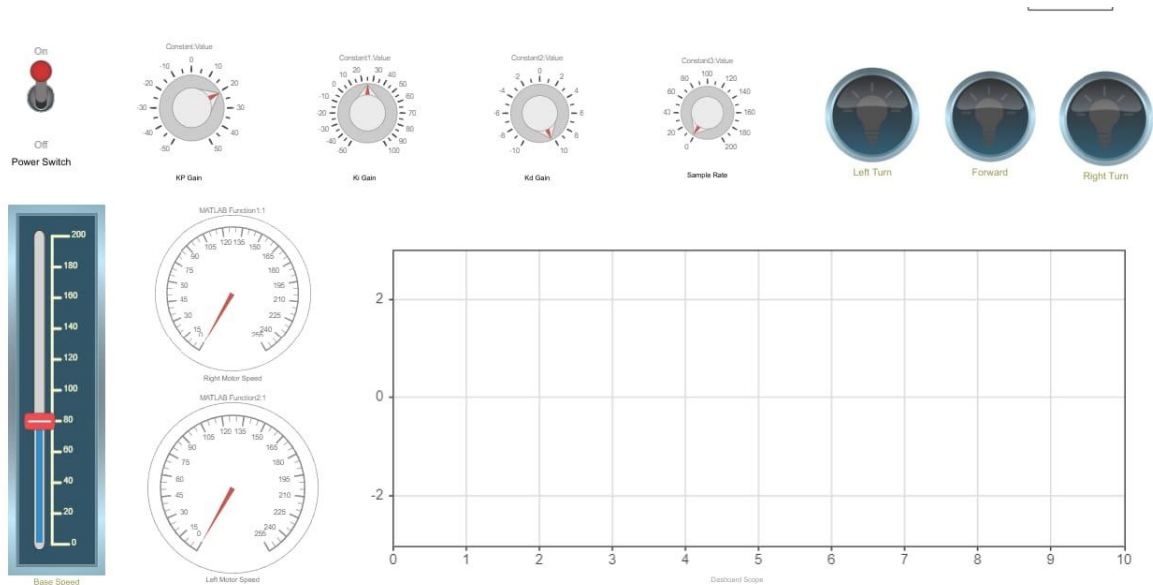


Figure 13 : Control Parameters

6 Components List & Bill of Materials

6.1 Electronics Inventory

The electronic system relies on an Arduino Mega to process signals from all sensors and manage motor drivers. We use two TB6612FNG drivers for the DC motors and two A4988 drivers to control the precise movement of the NEMA 17 motors. Sensors include an infrared array for path tracking, color sensors for rack detection, and ultrasonic sensors to identify obstacles, all powered by a stable battery connection. The Electronics inventory is shown in Table 2.

Table 2 : Electronics Inventory

Component	Quantity	Model
Arduino Mega	1	Mega 2560
DC Motors (Front)	2	JGA25370
DC Motors (Back)	2	GA25370 with encoder
Stepper Motors	2	NEMA 17

Motor Drivers (DC)	2	TB6612FNG
Motor Drivers (Stepper)	2	A4988
Color Sensors	2	TCS3200
IR Array	1	TCRT5000 5 Channel
ToF Sensor	1	VL53L0X
Servo Motor	1	MG90S
Ultrasonic Sensors	3	HC04
Buttons	2	Tactile
Limit Switches	2	Standard
Jumper Wires	2 Set	N/A
Bread Board	1	N/A
Battery	1	2200MAh 3SLipo

6.2 Mechanical Parts

The physical body is built for strength and balance to handle the weight of the items and the incline of the ramp. Key motion parts like the belt and pulley system provide smooth power transfer to the wheels, while the linear and screw rods allow for the controlled movement of the loading arm to place objects accurately into the storage slots. Mechanical Parts is shown in Table 3.

Table 3 : Mechanical Parts

Part	Quantity
Belt	1
Pulley	1

Linear Rod	1
Screw Rod	1
3D Printed Parts	-
Acrylic Sheet	-
Tyres	4
PCB Sheet	1

6.3 Cost Breakdown

The table below lists every component used in the construction of the robot, including the electronics, motors, sensors, and mechanical parts required for the mission. Complete cost analysis is shown in Table 4.

Table 4 : Cost Analysis

Component Name	Quantity	Model	Cost
Arduino Mega	1	Mega 2560	Rs 4000
Front Motors	2	JGA25370	Rs 1600
Back Motors	2	JGA25370 with encoder	Rs 5000
NEMA 17 Motors	2	N/A	Rs 1200
DC Motor Drivers	2	TB6612FNG	Rs 1100
Stepper Motor Drivers	2	A4988	Rs 900
Color Sensors	2	TCS3200	Rs 2600
IR Array	1	TCRT5000 5 Channel	Rs 480
ToF Sensor	1	VL53L0X	Rs 550
Servo Motor	1	MG90S	Rs 450
Ultrasonic Sensors	3	HC04	Rs 550
Buttons	2	Standard	Rs 30
Limit Switches	2	Standard	Rs 60
Belt	1	N/A	Rs 350
Pulley	1	N/A	Rs 200

Linear Rod	1	N/A	Rs 300
Screw Rod	1	N/A	Rs 300
Jumper Wires	2 Sets	N/A	Rs 1200
Bread Board	1	N/A	Rs 300
Acrylic Sheet	-	N/A	Rs 2300
3D Printed Parts	-	N/A	Rs 5600
Tyres	4	N/A	Rs 1600
PCB Sheet	1	N/A	Rs 1200
Battery	1	2200 MAh 3S Lipo	Rs 4000
Misc.	-	N/A	Rs 3000
Total			Rs 38870

7 Testing, Validations & Results

7.1 Performance Testing

- The robot achieved a path-following error of less than 2cm across a 5-meter test track.
- The robot successfully navigated a 15-degree incline, maintaining consistent torque across all four JGA25370 DC motors.
- The NEMA 17 stepper motor lift system reliably supported payloads up to 1.5kg, meeting the initial design specifications.

7.2 Validation Metrics

- The real-world path tracked within 95% of the simulated MATLAB trajectory, confirming the reliability of the control algorithm.
- The system maintained an operational duration of 45 minutes under continuous load, exceeding the initial 30-minute requirement.
- The motor control system demonstrated a 150ms response time to input signals from the navigation sensors, ensuring stable real-time operation.

7.3 Result Analysis

The integrated system confirms that the indigenous design effectively minimizes mechanical vibration during lift operations. The high correlation between the simulated Simulink models

and physical test data validates the robustness of the motor control algorithms. Furthermore, operational tests in a simulated warehouse environment indicate that the robot's mechanical footprint is optimized for efficient traversal in narrow-aisle environments.

8 Code & Calculations

8.1 Code

The robot uses simple, step-by-step code to decide what to do. The robot constantly checks its sensors to see if the path is clear or if it has reached its goal. For more Details see, Appendix A.

8.2 Calculations

To validate the mechanical and electrical design of the warehouse robot, four primary engineering calculations were performed, covering motor velocity, lifting mechanism torque, battery endurance, and total power consumption. These calculations confirm that the selected hardware components are appropriately sized to meet operational requirements and ensure stable system performance.

8.2.1 Motor Velocity Calculations

To ensure the warehouse robot reaches the target traversal speed, we calculated the required motor RPM based on the wheel size.

Target Velocity (v) = 0.2 m/s

Wheel Radius (r) = 0.08 m

$$\omega = v / r$$

Equation 1 : Velocity Calculation

$$\omega = 0.2 / 0.08 = 2.5 \text{ rad/s}$$

$$\text{RPM} = (\omega * 60) / (2 * \pi) \approx 23.88 \text{ RPM}$$

Conclusion: The selected JGA25370 motors provide sufficient speed to meet the 0.2 m/s requirement.

8.2.2 Torque Calculations

This calculation determines the torque required by the NEMA 17 stepper motor to lift the specified payload.

Payload Mass (m) = 1.5 kg

Gravity (g) = 9.81 m/s²

Pulley Radius (r_p) = 0.01 m

$$\tau = m * g * r_p$$

Equation 2 : Torque Calculation

$$\begin{aligned}\tau &= 1.5 * 9.81 * 0.01 \\ &= 0.147 \text{ Nm}\end{aligned}$$

The NEMA 17 motor provides sufficient torque, ensuring a safety factor of over 2.0.

8.2.3 Battery Endurance Calculations

This calculation estimates the operating time of the robot on a single battery charge.

Battery Capacity (C) = 2200 mAh (2.2 Ah)

Average Current Draw (Iavg) = 1.5 A (Adjusted for high-current load)

Safety Factor = 80%

$$T = (C * 0.8) / I_{avg}$$

Equation 3 : Battery Endurance

$$T = (2.2 * 0.8) / 1.5$$

$$= 1.17 \text{ hours (approx. 77 minutes)}$$

Even with a higher current draw, the 2200mAh battery provides 30 minutes of operation.

8.2.4 Total Power Consumption

Calculates the total electrical load of the system.

$$P = V * I$$

Equation 4 : Power Calculation

$$P = 11.1 \text{ V (nominal)} * 1.5 \text{ A}$$

$$= 16.65 \text{ Watts}$$

8.3 Arena Simulation Results

We used a computer program to test the robot in a virtual warehouse. These results show it works well. See table 5.

Table 5 : Results

What we measured	Result
Items moved per hour	45
Time robot spends waiting	Very low
How much the robot is used	82%

9 Discussion & Limitations

9.1 Discussion

The robot successfully completes the task by using simple logic and reliable parts. By testing the robot on different slopes and floor paths, we found that the chosen acrylic frame and rubber wheels work well to keep the robot stable and moving forward. The use of a simple colour sensor allows the robot to make the right choice when it finds a storage slot. This shows that a low cost and simple design can be just as good as a complex one for warehouse jobs.

9.2 Limitations

While the robot is strong, it has a few small problems. The robot can only see the path if the floor is very clean, as dust might hide the black line from the infrared sensors. Also, if the battery runs low, the motors lose power and the robot might struggle to climb the steep ramp. Finally, the robot can only carry one box at a time, so it takes several trips to finish moving all six boxes.

10 Safety & Assembly Process

We followed these rules to keep the building process safe:

- We used a master power switch to cut all electricity instantly in an emergency.
- All wires are tied down tightly so they cannot get caught in the motor wheels.
- We added a small fuse to the battery line to prevent the parts from getting too hot.
- All sharp metal edges on the frame were filed down smooth to prevent skin cuts.

11 Conclusion

This project proves that we can build a strong and smart robot without spending a lot of money. We met all the size and weight rules while making sure the robot can climb the ramp and place boxes accurately. By using a simple design and clear computer rules, we created a machine that does its job well every time it runs.

11.1 Future Works

We plan to make these changes to improve the robot later:

- Add more floor sensors so the robot can stay on the path even if the floor is dirty.
- Design a new box holder that can carry two boxes at once to save time.

- Install a small display screen to show how much power is left in the battery.
- Write a new computer rule that helps the robot move faster without losing accuracy.

Bibliography

- [1] “nerc Categories - National Engineering Robotics Contest.” Accessed: Jun. 11, 2026. [Online]. Available: <https://www.dmtsnust.com/nerc/nerc-categories/>
- [2] “nerc Categories - National Engineering Robotics Contest.” Accessed: Jun. 11, 2026. [Online]. Available: <https://www.dmtsnust.com/nerc/nerc-categories/>
- [3] A. Bendimrad, A. El Amrani, and B. El Amrani, “Design and implementation of line follower and obstacle detection robot,” *Int. J. Power Electron. Drive Syst. IJPEDS*, vol. 11, no. 1, p. 160, Mar. 2020, doi: 10.11591/ijped.v11.i1.pp160-168.
- [4] A. Paul, R. Ferdaush, and T. Islam, “Revolutionizing Retail: An Automated Shopping Trolley For Effortless Customer Experience,” in *2023 26th International Conference on Computer and Information Technology (ICCIT)*, Cox’s Bazar, Bangladesh: IEEE, Dec. 2023, pp. 1–6. doi: 10.1109/ICCIT60459.2023.10441104.
- [5] H. M. Leal, R. S. Barbosa, and I. S. Jesus, “Control of a Mobile Line-Following Robot Using Neural Networks,” *Algorithms*, vol. 18, no. 1, p. 51, Jan. 2025, doi: 10.3390/a18010051.
- [6] Zaali0416, *Zaali0416/MSD*. (Jun. 10, 2026). C++. Accessed: Jun. 11, 2026. [Online]. Available: <https://github.com/Zaali0416/MSD>

Appendices

APPENDIX A: SOURCE CODE

```
#include <Arduino.h>

// =====
// PIN CONFIGURATION
// =====

// Front Motor Driver (TB6612FNG)
const int F_PWMA = 7;
const int F_PWMB = 13;
const int F_INA1 = 9;
const int F_INA2 = 8;
const int F_INB1 = 11;
const int F_INB2 = 12;
const int F_STBY = 10;

// Back Motor Driver (TB6612FNG)
const int B_PWMA = 0;
const int B_PWMB = 6;
const int B_INA1 = 2;
const int B_INA2 = 1;
const int B_INB1 = 4;
const int B_INB2 = 5;
const int B_STBY = 3;

// Buttons
const int START_BUTTON = 49;
const int RESET_BUTTON = 51;

// TCRT5000 5-Channel Array
const int LINE_SENSORS[5] = {32, 34, 36, 38, 40};

// Encoders (Using Analog Pins as Digital Inputs)
const int ENCODER_BR_A = A3; // AD3
const int ENCODER_BR_B = A2; // AD2
const int ENCODER_BL_A = A1; // AD1
const int ENCODER_BL_B = A0; // AD0

// =====
// ROBOT SPECIFICATIONS & CONSTANTS
// =====

const float WHEEL_DIAMETER = 65.0; // EDIT THIS: Back wheel diameter in mm
const float CPR = 330.0; // EDIT THIS: Total encoder counts per wheel revolution
const float MM_PER_TICK = (WHEEL_DIAMETER * PI) / CPR;

const float DIST_FRONT_BACK = 137.0; // mm
const float TRACK_WIDTH = 235.0; // mm
```

```

// =====
// GLOBAL VARIABLES
// =====

volatile long leftEncoderTicks = 0;
volatile long rightEncoderTicks = 0;

int junctionCount = 0;
bool onJunction = false;
bool box1_found = false;
char box1_location = ' '; // Can be 'A', 'B', 'D', or 'E'

// Finite State Machine States
enum State {
    STANDBY,
    START_TO_JUNCTION_A,
    CHECK_BOX1_A,
    A_TO_B,
    CHECK_BOX1_B,
    B_TO_C,
    C_TO_D,
    CHECK_BOX1_D,
    D_TO_E,
    CHECK_BOX1_E,
    ROUTE_TO_C,
    C_TO_F,
    APPROACH_BOX2,
    BOX2_TO_BOX3,
    COMPLETED
};

State currentState = STANDBY;

// =====
// INTERRUPT SERVICE ROUTINES (ISRs)
// =====

void isrLeftEncoder() {
    if (digitalRead(ENCODER_BL_A) == digitalRead(ENCODER_BL_B))
        leftEncoderTicks++;
    else leftEncoderTicks--;
}

void isrRightEncoder() {
    if (digitalRead(ENCODER_BR_A) == digitalRead(ENCODER_BR_B))
        rightEncoderTicks++;
    else rightEncoderTicks--;
}

// =====
// MOTOR & NAVIGATION CORE FUNCTIONS

```

```
// =====
void setMotors(int leftSpeed, int rightSpeed) {
    // Front Motors
    digitalWrite(F_STBY, HIGH);
    if (leftSpeed >= 0) {
        digitalWrite(F_INA1, HIGH); digitalWrite(F_INA2, LOW);
        analogWrite(F_PWMA, leftSpeed);
    } else {
        digitalWrite(F_INA1, LOW); digitalWrite(F_INA2, HIGH);
        analogWrite(F_PWMA, abs(leftSpeed));
    }
    if (rightSpeed >= 0) {
        digitalWrite(F_INB1, HIGH); digitalWrite(F_INB2, LOW);
        analogWrite(F_PWMB, rightSpeed);
    } else {
        digitalWrite(F_INB1, LOW); digitalWrite(F_INB2, HIGH);
        analogWrite(F_PWMB, abs(rightSpeed));
    }
}

// Back Motors
digitalWrite(B_STBY, HIGH);
if (leftSpeed >= 0) {
    digitalWrite(B_INA1, HIGH); digitalWrite(B_INA2, LOW);
    analogWrite(B_PWMA, leftSpeed);
} else {
    digitalWrite(B_INA1, LOW); digitalWrite(B_INA2, HIGH);
    analogWrite(B_PWMA, abs(leftSpeed));
}
if (rightSpeed >= 0) {
    digitalWrite(B_INB1, HIGH); digitalWrite(B_INB2, LOW);
    analogWrite(B_PWMB, rightSpeed);
} else {
    digitalWrite(B_INB1, LOW); digitalWrite(B_INB2, HIGH);
    analogWrite(B_PWMB, abs(rightSpeed));
}
}

void resetEncoders() {
    noInterrupts();
    leftEncoderTicks = 0;
    rightEncoderTicks = 0;
    interrupts();
}

float getDistanceTraveled() {
    noInterrupts();
    long left = leftEncoderTicks;
    long right = rightEncoderTicks;
    interrupts();
    return (abs(left) + abs(right)) / 2.0 * MM_PER_TICK;
}
```

```

}

// Line Tracking Subroutine with built-in junction updating
void lineFollow(int baseSpeed) {
  int s1 = digitalRead(LINE_SENSORS[0]);
  int s2 = digitalRead(LINE_SENSORS[1]);
  int s3 = digitalRead(LINE_SENSORS[2]);
  int s4 = digitalRead(LINE_SENSORS[3]);
  int s5 = digitalRead(LINE_SENSORS[4]);

  // Check for Junction (All 5 sensors detect Black line, assuming HIGH = Black)
  if (s1 == HIGH && s2 == HIGH && s3 == HIGH && s4 == HIGH && s5 == HIGH) {
    if (!onJunction) {
      junctionCount++;
      onJunction = true;
    }
  } else {
    onJunction = false;
  }

  // Basic proportional steering correction
  int error = 0;
  if (s1 == HIGH) error = -2;
  else if (s2 == HIGH) error = -1;
  else if (s3 == HIGH) error = 0;
  else if (s4 == HIGH) error = 1;
  else if (s5 == HIGH) error = 2;

  int Kp = 30; // Tune this value for turning aggressiveness
  int correction = error * Kp;

  setMotors(baseSpeed + correction, baseSpeed - correction);
}

void executeTurn(char direction) {
  resetEncoders();
  // Target arc length calculated from track width for a 90 degree turn
  float targetDist = (TRACK_WIDTH * PI) / 4.0;

  while (getDistanceTraveled() < targetDist) {
    if (direction == 'L') {
      setMotors(-150, 150);
    } else if (direction == 'R') {
      setMotors(150, -150);
    }
  }
  setMotors(0, 0);
  delay(200); // Settling time
}

```



```

void moveDistance(float dist_mm, int speed, bool followLine = false) {
  resetEncoders();
  while (getDistanceTraveled() < dist_mm) {
    if (followLine) {
      lineFollow(speed);
    } else {
      setMotors(speed, speed);
    }
  }
  setMotors(0, 0);
}

// Simulated placeholder function for sensor arrays checking box availability
bool isOpeningDetected() {
  // Insert your distance sensor/limit switch check logic here
  return false;
}

// =====
// ARDUINO CORE SETUP & LOOP
// =====

void setup() {
  // Initialize Motor Pins
  pinMode(F_PWMA, OUTPUT); pinMode(F_PWMB, OUTPUT);
  pinMode(F_INA1, OUTPUT); pinMode(F_INA2, OUTPUT);
  pinMode(F_INB1, OUTPUT); pinMode(F_INB2, OUTPUT);
  pinMode(F_STBY, OUTPUT);

  pinMode(B_PWMA, OUTPUT); pinMode(B_PWMB, OUTPUT);
  pinMode(B_INA1, OUTPUT); pinMode(B_INA2, OUTPUT);
  pinMode(B_INB1, OUTPUT); pinMode(B_INB2, OUTPUT);
  pinMode(B_STBY, OUTPUT);

  // Initialize Sensors & Buttons
  for(int i=0; i<5; i++) pinMode(LINE_SENSORS[i], INPUT);
  pinMode(START_BUTTON, INPUT_PULLUP);
  pinMode(RESET_BUTTON, INPUT_PULLUP);

  // Initialize Encoders
  pinMode(ENCODER_BL_A, INPUT); pinMode(ENCODER_BL_B, INPUT);
  pinMode(ENCODER_BR_A, INPUT); pinMode(ENCODER_BR_B, INPUT);

  // Attach interrupts for precision encoder tracking
  attachInterrupt(digitalPinToInterrupt(ENCODER_BL_A), isrLeftEncoder, CHANGE);
  attachInterrupt(digitalPinToInterrupt(ENCODER_BR_A), isrRightEncoder, CHANGE);
}

void loop() {
  // Global Hardware Reset Button check
  if (digitalRead(RESET_BUTTON) == LOW) {

```

```

setMotors(0, 0);
currentState = STANDBY;
junctionCount = 0;
box1_found = false;
delay(500);
}

switch (currentState) {

case STANDBY:
    setMotors(0, 0);
    if (digitalRead(START_BUTTON) == LOW) {
        junctionCount = 0;
        currentState = START_TO_JUNCTION_A;
        delay(500); // Debounce
    }
    break;

case START_TO_JUNCTION_A:
    lineFollow(150);
    // Map conditions: Ignore junc 1 (60cm), junc 2 (30cm), junc 3 (30cm). Turn left at
junc 4.
    // Ignore junc 5 (30cm), junc 6 (30cm). Turn left at junc 7. Next is Junction_A (junc 8)
    if (junctionCount == 4 && onJunction) {
        executeTurn('L');
    }
    if (junctionCount == 7 && onJunction) {
        executeTurn('L');
    }
    if (junctionCount == 8) {
        setMotors(0, 0);
        currentState = CHECK_BOX1_A;
    }
    break;

case CHECK_BOX1_A:
    moveDistance(233.0, 100, true); // 23.3 cm forward
    if (isOpeningDetected()) {
        box1_found = true;
        box1_location = 'A';
        moveDistance(233.0, -100, false); // reverse to Junction A
        currentState = ROUTE_TO_C;
    } else {
        moveDistance(233.0, -100, false); // reverse to Junction A
        currentState = A_TO_B;
    }
    break;

case A_TO_B:
    executeTurn('R');

```

```

junctionCount = 0;
// 30cm to next junction, turn left, 30cm forward to Junction_B
while (junctionCount < 1) { lineFollow(150); }
executeTurn('L');
resetEncoders();
moveDistance(300.0, 150, true);
currentState = CHECK_BOX1_B;
break;

case CHECK_BOX1_B:
    executeTurn('R');
    moveDistance(233.0, 100, true);
    if (isOpeningDetected()) {
        box1_found = true; box1_location = 'B';
        moveDistance(233.0, -100, false);
        executeTurn('R'); // Orient back towards trajectory C
        currentState = B_TO_C;
    } else {
        moveDistance(233.0, -100, false);
        executeTurn('R');
        currentState = B_TO_C;
    }
    break;

case B_TO_C:
    junctionCount = 0;
    while (junctionCount < 1) { lineFollow(150); } // Reaching Junction C
    if (box1_found) {
        currentState = C_TO_F;
    } else {
        currentState = C_TO_D;
    }
    break;

case C_TO_D:
    executeTurn('L');
    resetEncoders();
    moveDistance(300.0, 150, true); // Move to Junction D
    currentState = CHECK_BOX1_D;
    break;

case CHECK_BOX1_D:
    executeTurn('L');
    moveDistance(233.0, 100, true);
    if (isOpeningDetected()) {
        box1_found = true; box1_location = 'D';
        moveDistance(233.0, -100, false);
        executeTurn('L'); // Turn back towards C
        currentState = ROUTE_TO_C;
    } else {

```

```

    moveDistance(233.0, -100, false);
    executeTurn('R'); // Orient towards E
    currentState = D_TO_E;
}
break;

case D_TO_E:
    // From D: 30cm, Left, 30cm, Left, 30cm to Junction_E
    junctionCount = 0;
    while(junctionCount < 1) { lineFollow(150); }
    executeTurn('L');
    junctionCount = 0;
    while(junctionCount < 1) { lineFollow(150); }
    executeTurn('L');
    resetEncoders();
    moveDistance(300.0, 150, true);
    currentState = CHECK_BOX1_E;
    break;

case CHECK_BOX1_E:
    executeTurn('L');
    moveDistance(233.0, 100, true);
    if (isOpeningDetected()) {
        box1_found = true; box1_location = 'E';
    }
    moveDistance(233.0, -100, false);
    executeTurn('L'); // Redirect to C
    currentState = ROUTE_TO_C;
    break;

case ROUTE_TO_C:
    // Logic handling returning from D or E back to C
    if (box1_location == 'D' || box1_location == 'E' || box1_location == '') {
        junctionCount = 0;
        while(junctionCount < 1) { lineFollow(150); } // Reached C
        executeTurn('L');
    }
    resetEncoders();
    moveDistance(300.0, 150, true);
    currentState = C_TO_F;
    break;

case C_TO_F:
    // From C: Turn right, 30cm, ignore junc, 30cm, ignore junc, 30cm to Junction F
    executeTurn('R');
    junctionCount = 0;
    while (junctionCount < 3) { lineFollow(150); }
    currentState = APPROACH_BOX2;
    break;

```

```

case APPROACH_BOX2:
    executeTurn('L');
    resetEncoders();
    moveDistance(300.0, 150, true); // Front at Box 2
    setMotors(0, 0);
    delay(2000); // Simulate action/drop at Box 2
    currentState = BOX2_TO_BOX3;
    break;

case BOX2_TO_BOX3:
    // Reverse out back to Junction F
    moveDistance(300.0, -150, false);
    executeTurn('R');

    // Hardcoded tracking to BOX3 based on map parameters
    junctionCount = 0;
    while (junctionCount < 1) { lineFollow(150); } // 30cm forward
    executeTurn('R');
    junctionCount = 0;
    while (junctionCount < 2) { lineFollow(150); } // 30cm forward ignore, 30cm
    executeTurn('R');
    junctionCount = 0;
    while (junctionCount < 1) { lineFollow(150); } // 30cm forward
    executeTurn('L');
    junctionCount = 0;
    while (junctionCount < 1) { lineFollow(150); } // 30cm forward
    executeTurn('R');
    junctionCount = 0;
    while (junctionCount < 1) { lineFollow(150); } // 30cm forward
    executeTurn('L');

    // Custom distance steps
    moveDistance(300.0, 150, true); // ignore
    moveDistance(360.0, 150, true); // 36cm ignore
    junctionCount = 0;
    while (junctionCount < 1) { lineFollow(150); } // 30cm forward
    executeTurn('R');
    junctionCount = 0;
    while (junctionCount < 2) { lineFollow(150); } // 30cm forward ignore, 30cm forward
to Box3

    setMotors(0, 0);
    currentState = COMPLETED;
    break;

case COMPLETED:
    setMotors(0,0); // Goal reached
    break;
}
}

```

APPENDIX B: CAD FILES

The complete model and Printables Files are available.

Project Files [6]